

Diátaxis Material for MkDocs Guide

Welcome to a complete, practical documentation project for **Material for MkDocs**, organized using the **Diátaxis** framework.

Source code

The source code for this documentation is available [here](#)

What is MkDocs

[MkDocs](#) is a static site generator designed for project documentation. It builds a website from Markdown files and a simple configuration file, making it easy to create and maintain documentation sites.

What is Material for MkDocs

[Material for MkDocs](#) is a popular theme and feature-rich extension for MkDocs. It adds a modern design, navigation improvements, search, and many useful documentation-focused features.

What is Diátaxis

[Diátaxis](#) is a documentation framework that organizes content into four distinct types: tutorials, how-to guides, reference, and explanation. It helps readers choose the right page for their current goal.

Overview

As the Diátaxis framework has four quadrants, this documentation is organized accordingly.

Tutorials

 2026-05-09  GitHub 

Tutorials

 [Getting started](#)

 [First site](#)

 2026-05-09  GitHub 

Getting started

This guide explains how to install Material for MkDocs. You will set up a Python environment, install the required packages, and verify that the `mkdocs` command is ready to use.

Prerequisites

- Python 3.8+
- `pip`

Create a virtual environment first

Using a virtual environment is highly recommended to isolate project dependencies.

Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows

```
python.exe -m venv .venv
.venv\Scripts\activate
```

Install Material for MkDocs

The `mkdocs-material` package includes MkDocs, Material for MkDocs, and all of their dependencies. Install it from PyPI using `pip`:

```
pip install mkdocs-material
```

Verify the installation

Check that the `mkdocs` command is available and see which version you installed:

```
mkdocs --version
```



Fixing live reload

A recent dependency update in MkDocs can prevent the live-reloading from working. This is a known issue (see [#4032](#)). If you run `mkdocs serve` and your browser does not automatically refresh after you change a file, you most likely need to downgrade the `click` package:

```
pip install "click==8.2.1"
```

You are now ready to create your first site.



2026-05-09



GitHub



Your first site

This tutorial guides you through creating a new documentation site, starting the local development server, editing a page, and building the static output.

Create a new project

The `mkdocs new` command creates a new project directory with a default `mkdocs.yml` file and a single `index.md` page.

```
mkdocs new my-project
cd my-project
```

Your folder now contains:

```
my-project/
├── docs/
│   └── index.md
└── mkdocs.yml
```

- `mkdocs.yml` : This is your main configuration file. You'll use it to set your site's title, choose a theme, and configure navigation.
- `docs/` : This directory contains all of your documentation content, written as Markdown files. `index.md` is the default home page.

Start the local server

The `mkdocs serve` command starts a local development server that watches for file changes and automatically rebuilds your site.

```
mkdocs serve
```

Open `http://127.0.0.1:8000` in your browser.

✓ Expected result

You should see a simple documentation site with a single home page.

Edit your first page

Open `docs/index.md` in your editor and replace the default content:

docs/index.md

```
# Hello, world!  
  
This is my first MkDocs site.
```

When you save the file, the browser will automatically reload to show the new content.

💧 What to notice

The page updates immediately because the development server watches your files for changes.

Build the static site

When you are ready to publish, use the `mkdocs build` command to generate the static HTML, CSS, and JavaScript files.

```
mkdocs build
```

This creates a `site/` directory containing the complete, self-contained website.

You have now created, previewed, and built your first documentation site.

Adding pages and navigation

This tutorial shows you how to add new pages to your site and organize them into a structured navigation menu.

Create new pages

First, create a few new Markdown files inside your `docs` directory. For better organization, it's a good practice to group related pages into sub-folders.

Create a new folder `docs/features` and add two files:

`docs/features/search.md`:

```
# Search

Our documentation has built-in search.
```

`docs/features/themes.md`:

```
# Themes

We use the Material for MkDocs theme.
```

Your project structure should now look like this:

```
my-project/
├── docs/
│   ├── features/
│   │   ├── search.md
│   │   └── themes.md
│   └── index.md
└── mkdocs.yml
```

If your `mkdocs serve` process is still running, you will notice that these new pages are not yet visible in the site's navigation.

Configure the navigation

To add the new pages to your site's navigation, you need to edit the `nav` section in your `mkdocs.yml` file.

Open `mkdocs.yml` and add the `nav` configuration:

`mkdocs.yml`

```
site_name: My Docs
nav:
  - Home: index.md
  - Features:
    - Search: features/search.md
    - Themes: features/themes.md
```

What this does

- `nav` : This top-level key tells MkDocs that you want to define a manual navigation structure.
- `Home: index.md` : This creates a top-level link labeled "Home" that points to your `index.md` file.
- `Features:` : This creates a top-level navigation section labeled "Features".
- The indented list under `Features` creates a dropdown menu with links to your new pages.

✓ Expected result

Your site now has a "Features" dropdown menu in the navigation bar, containing links to your "Search" and "Themes" pages.

You have now learned how to expand your site with new pages and control their order and structure in the navigation.

Configuring your site

This tutorial explains how to configure your project by editing the `mkdocs.yml` file. You will learn how to set basic site information, apply the Material for MkDocs theme, and enable some of its key features.

Set basic site information

Let's configure the site by setting these core options in `mkdocs.yml`. Update your file to look like this:

`mkdocs.yml`

```
site_name: My Docs
site_url: https://example.com/
site_description: A short description of my documentation site.
repo_url: https://github.com/your-username/your-repo

nav:
  - Home: index.md
```

- `site_name`: The title of your documentation site.
- `site_url`: The full public URL where your site will be hosted.
- `site_description`: A short summary used for search engine results and metadata.
- `repo_url`: The URL of your source code repository (e.g., GitHub, GitLab).

Apply the Material for MkDocs theme

Now, let's switch from the default theme to Material for MkDocs.

Add the `theme` section to your `mkdocs.yml`:

`mkdocs.yml`

```
site_name: My Docs
# ... (other settings)

theme:
  name: material
```

✓ Expected result

If `mkdocs serve` is running, your site will automatically reload with a completely new look and feel. You are now using the Material for MkDocs theme.

Enable theme features

Material for MkDocs includes many features that can be enabled under the `theme.features` key. Let's add a few popular ones.

Update your `theme` configuration:

```
mkdocs.yml

# ... (other settings)

theme:
  name: material
  features:
    - navigation.tabs
    - navigation.footer
    - navigation.top
```

- `navigation.tabs`: Turns the top-level navigation into tabs.
- `navigation.footer`: Adds "Previous" and "Next" page links in the footer.
- `navigation.top`: Adds a "Back to top" button that appears when you scroll down.

Enable and configure search

Search is a built-in plugin that is enabled by default. However, as soon as you add the `plugins` key to your configuration, you must explicitly include `search` to keep it.

Add the `plugins` section to your `mkdocs.yml`:

mkdocs.yml

```
# ... (other settings)
```

```
plugins:  
  - search:
```

- `plugins`: The list of plugins to enable.

Change the color palette

You can easily change the site's colors using the `palette` option.

Update your `theme` configuration again:

mkdocs.yml

```
# ... (other settings)
```

```
theme:  
  name: material  
  features:  
    - navigation.tabs  
    - navigation.footer  
    - navigation.top
```

```
palette:  
  scheme: slate  
  primary: blue
```

```
plugins:  
  - search:
```

- `scheme: slate`: Switches the site to a dark mode theme.
- `primary: blue`: Sets the primary accent color to blue.

You have now learned the basics of configuring your site, applying a theme, and customizing its features and appearance.

Markdown extensions

This tutorial explains how to use some of Material for MkDocs' most useful Markdown extensions to make your content more readable and interactive. You will learn how to add admonitions, styled code blocks, content tabs, and collapsible blocks.

Prerequisites

To use these extensions, enable the corresponding Markdown extensions in your `mkdocs.yml` file. Make sure your `markdown_extensions` section looks like this:

mkdocs.yml

```
markdown_extensions:
  - admonition
  - pymdownx.details
  - pymdownx.superfences
  - pymdownx.tabbed:
      alternate_style: true
```

Admonitions

Admonitions are styled call-out blocks that are perfect for notes, warnings, or tips.

How to use them

Create an admonition by using `!!!` followed by a type qualifier.

```
!!! note
    This is a note. It contains information that is useful but not critical.

!!! warning "Don't forget"
    This is a warning. It contains important information that the user should not ignore.
```

Example:

Quick fix

This is a tip.

Common types

- `note`
- `info`
- `tip`
- `question`
- `warning`
- `danger`
- `bug`
- `example`

Code blocks

You can add syntax highlighting and titles to your code blocks.

How to use them

To enable syntax highlighting, add the language identifier after the opening backticks. To add a title, use `title="Your Title"` after the language.

```
```python title="my_script.py"
import os

def main():
 print("Hello, world!")

if __name__ == "__main__":
 main()
```

Example:

**my\_script.py**

```
import os

def main():
 print("Hello, world!")

if __name__ == "__main__":
 main()
```

## Content tabs

Content tabs are useful for grouping related but distinct information, such as installation instructions for different operating systems.

### How to use them

Use `=== "Tab Title"` to create each tab.

```
=== "Linux / macOS"
 ``bash
 python3 -m venv .venv
 source .venv/bin/activate
 ``

=== "Windows"
 ``powershell
 python.exe -m venv .venv
 .venv\Scripts\activate
 ``
```

Example:

#### Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

#### Windows

```
python.exe -m venv .venv
.venv\Scripts\activate
```

## Collapsible blocks

You can hide less critical or lengthy content inside collapsible "details" blocks.

## How to use them

Use `???` for a block that is closed by default or `??#+` for a block that is open by default.

```
??? note "Click to expand"
```

```
 This content is hidden by default. It's useful for supplementary information
 or long log outputs.
```

```
??#+ tip "This one is open"
```

```
 This content is visible by default but can be collapsed by the user.
```

Example:

 **Build log (collapsed)** 

```
INFO Building documentation...
DEBUG Reading file: docs/index.md
ERROR Missing reference: api/usage.md
```

You can now use these extensions to create richer and more user-friendly documentation pages.



# How-To



[Generate a PDF file out of MkDocs](#)



[Deploy to GitHub Pages](#)



[Dark/Light Mode Toggle](#)



[Show information about pages](#)



2026-05-09



GitHub



# Deploy MkDocs to Github Pages

This page describes how to deploy your MkDocs documentation to Github Pages.

## Github pages

[Github Pages](#) is a static site hosting service that allows you to publish your documentation directly from a GitHub repository.

### Plan Availability and Usage

GitHub Pages is available for free in public repositories. GitHub Pages is not allowed for commercial use. See [GitHub Pages limits](#) for more details.

## Deploying to Github Pages

To automate your MkDocs deployment to GitHub Pages, you can use GitHub Actions. This workflow automatically builds and deploys your documentation whenever you push changes to your repository.

## Setting up GitHub Actions

Create a new workflow file at `.github/workflows/deploy-docs.yml` in your repository with the following content:

```

name: deploy-docs

on: # (1)
 workflow_dispatch:
 push:
 branches:
 - master
 paths:
 - 'docs/**'
 - 'mkdocs.yml'

permissions:
 contents: write

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - name: Git Credentials
 run: |
 git config user.name github-actions[bot]
 git config user.email 41898282+github-
actions[bot]@users.noreply.github.com
 - uses: actions/setup-python@v5
 with:
 python-version: 3.x
 - run: echo "cache_id=$(date --utc '+%V')" >> $GITHUB_ENV
 - uses: actions/cache@v4
 with:
 key: mkdocs-material-${{ env.cache_id }}
 path: ~/.cache
 restore-keys: |
 mkdocs-material-
 - run: pip install -r requirements.txt
 - run: mkdocs gh-deploy --force

```

1. It can be freely chosen, when to trigger the workflow

## Enabling GitHub Pages

After the first successful workflow run, you need to configure your repository:

1. Go to your repository **Settings** → **Pages**
2. Under "Build and deployment", select "Deploy from a branch"
3. Choose `gh-pages` as the branch and `/ (root)` as the folder

Your documentation will now be automatically deployed to GitHub Pages at

`https://<username>.github.io/<repository>/` whenever you push changes to the `master` branch.

## How it works

Any trigger, whether manual or caused by a push, starts the workflow. MkDocs then builds the documentation. After the build, the workflow pushes the generated site to the `gh-pages` branch, which is why write permission for repository contents is required. Finally, GitHub Pages serves the site from that branch.



2026-05-09



GitHub



# Generate a PDF file out of MkDocs

This page describes how to generate a PDF file from your MkDocs documentation using `mkdocs-exporter`.

## PDF Export from MkDocs

MkDocs does not natively support PDF generation. However, the MkDocs plugin ecosystem provides several solutions to export your documentation as PDF. Popular plugins include `mkdocs-pdf-export-plugin` and `mkdocs-with-pdf`, However this how-to focuses on `mkdocs-exporter`.

## Installing mkdocs-exporter

First, install the `mkdocs-exporter` package using `pip`:

```
pip install mkdocs-exporter
```

Additionally, Playwright needs to install browser binaries that it uses to render pages. Run the following to install those browsers:

```
playwright install
```

## Configuring mkdocs-exporter

Add the plugin to your `mkdocs.yml` configuration file with the basic settings:

```
plugins:
 - exporter:
 formats:
 pdf:
 enabled: true
 concurrency: 8
```

The `concurrency` setting controls how many pages are rendered in parallel, speeding up the PDF generation process.

## Aggregating into a Single PDF

By default, the exporter generates individual PDF files for each page. If you want to combine all pages into a single PDF document, enable the aggregator option:

```
plugins:
 - exporter:
 formats:
 pdf:
 enabled: true
 concurrency: 8
 aggregator:
 enabled: true
 covers: all
 output: output.pdf
```

## Generating the PDF

After configuration, generate the PDF by running:

```
mkdocs build
```

For the configured aggregation, the combined PDF will be located at `site/output.pdf`. Individual page PDFs are also generated and placed in the same `site/` directory alongside your static HTML files.

# Implementing a Dark/Light Mode Toggle

This page describes how to add a dark and light mode toggle to your Material for MkDocs documentation.

## Color Scheme Preferences

Material for MkDocs supports multiple color schemes, allowing users to choose between light and dark modes based on their preference. You can configure these schemes to automatically respect the users system preference while providing a manual toggle to switch between them.

## Setting up the Color Scheme Toggle

To add a dark and light mode toggle to your documentation, configure the `palette` section in your `mkdocs.yml` file. This section defines the available color schemes and how users can switch between them.

### Basic Configuration

Add the palette configuration to your `theme` section in `mkdocs.yml`:

```
theme:
 name: material

palette:
 - media: "(prefers-color-scheme: light)"
 scheme: default
 primary: white
 accent: blue
 toggle:
 icon: material/weather-night
 name: Dark mode
 - media: "(prefers-color-scheme: dark)"
 scheme: slate
 primary: black
 accent: blue
 toggle:
 icon: material/weather-sunny
 name: Light mode
```



# Show information about a page

This page shows how to add page information to your MkDocs site with two plugins: one for showing who edited a page and one for showing when it was last changed.

## Show who edited the page

Use [mkdocs-git-committers-plugin-2](#) to display the contributors who edited a page. The information will be displayed in the footer of the page.

Add the plugin to `mkdocs.yml` like this:

```
plugins:
- git-committers:
 repository: khde/diataxis-material-mkdocs-guide
 branch: master
 cache_dir: .cache/plugin/git-committers/
```

The plugin simply reads the Git history and shows the people who contributed to the page.

## Show when the page was last edited

Use [mkdocs-git-revision-date-localized-plugin](#) to show the last edit date on each page.

Add it to `mkdocs.yml` like this:

```
plugins:
- git-revision-date-localized:
 locale: en
 timezone: Europe/Amsterdam
 type: timeago
```

# Reference

> [Commands](#)

 [YAML Configuration](#)

 2026-05-09  GitHub 

# Command reference

This page provides a quick reference to the main MkDocs commands. For the complete set of options, refer to your local help output: `mkdocs --help` and `mkdocs <command> --help`.

## `mkdocs new`

Creates a new project directory with a minimal `mkdocs.yml` and a starter docs page.

```
mkdocs new my-project
```

## `mkdocs serve`

Runs a local development server and rebuilds the site on file change.

```
mkdocs serve
```

## `mkdocs build`

Builds the static site output (defaults to the `site/` directory).

```
mkdocs build
```

## `mkdocs gh-deploy`

Builds the site and publishes it to a Git branch, intended for GitHub Pages.

```
mkdocs gh-deploy
```

For a detailed how-to on publishing MkDocs sites to GitHub Pages, refer to this guide [here](#).

YAML configuration

Reference configuration

```
site_name: Your Project Docs # (1)
site_description: My description here # (2)
site_url: https://<your-user>.github.io/<your-repo>/ # (3)
repo_url: https://github.com/<your-user>/<your-repo>/ # (4)
edit_uri: edit/main/docs/ # (5)

docs_dir: docs # (6)

nav:
 - Home: index.md # (7)
 - Getting started:
 - getting-started/index.md # (8)
 - Installation: getting-started/installation.md
 - Usage: getting-started/usage.md
 - About: about.md

theme:
 name: material # (9)
 icon:
 repo: fontawesome/brands/github # (10)
 logo: material/book-open-page-variant # (11)

features:
 - content.action.view # (12)
 - content.code.copy # (13)
 - content.code.annotate # (14)
 - navigation.footer # (15)
 - navigation.top # (16)
 - navigation.indexes # (17)
 - navigation.tabs # (18)
 - navigation.tabs.sticky # (19)
 - search.suggest # (20)
 - search.highlight # (21)
 - search.share # (22)

palette:
 - media: "(prefers-color-scheme: light)" # (23)
 scheme: default
 primary: white
 accent: blue
 toggle:
 icon: material/weather-night
 name: Dark mode
 - media: "(prefers-color-scheme: dark)" # (24)
 scheme: slate
 primary: black
 accent: blue
 toggle:
 icon: material/weather-sunny
 name: Light mode

font:
 text: Inter # (25)
```

```
code: Google Sans Code # (26)markdown_extensions: - admonition # (27) -
attr_list # (28) - md_in_html # (29) - pymdownx.details # (30) -
pymdownx.superfences # (31)plugins: - search # (32)extra: generator: false #
(33)
```

1. `site_name`: Sets the site's main title, appearing in the header and browser tab.
2. `site_description`: Provides a brief summary for search engine results and metadata.
3. `site_url`: Defines the canonical public URL for the site.
4. `repo_url`: Links to the source code repository, often displayed as an icon.
5. `edit_uri`: Constructs the "Edit this page" link for each page.
6. `docs_dir`: Specifies the directory containing the Markdown source files.
7. `nav`: Defines the site's navigation structure and page order.
8. Section `index.md` pages serve as stable, linkable landing pages for navigation sections.
9. `theme.name`: Specifies the theme to use for the site's appearance.
10. `theme.icon.repo`: Sets the icon for the repository link.
11. `theme.icon.logo`: Sets the site's logo icon, displayed in the header.
12. `features.content.action.view`: Adds a button to view the source of the current page.
13. `features.content.code.copy`: Adds a copy button to all code blocks.
14. `features.content.code.annotate`: Enables code annotations for explaining code snippets.
15. `features.navigation.footer`: Adds previous/next page links in the footer.
16. `features.navigation.top`: Adds a "back to top" button.
17. `features.navigation.indexes`: Makes section index pages act as landing pages in the navigation.
18. `features.navigation.tabs`: Renders top-level navigation items as tabs.
19. `features.navigation.tabs.sticky`: Makes the navigation tabs stick to the top of the viewport.
20. `features.search.suggest`: Provides search query auto-completion.
21. `features.search.highlight`: Highlights search terms on result pages.
22. `features.search.share`: Adds a button to share a search query.
23. `palette (light)`: Defines the color scheme for light mode.
24. `palette (dark)`: Defines the color scheme for dark mode.
25. `font.text`: Sets the main font for body text.

- 26. `font.code` : Sets the font for code blocks.
- 27. `markdown_extensions.admonition` : Enables styled blocks for notes, warnings, and tips.
- 28. `markdown_extensions.attr_list` : Allows adding HTML attributes to Markdown elements.
- 29. `markdown_extensions.md_in_html` : Allows processing Markdown syntax inside HTML blocks.
- 30. `markdown_extensions.pydownx.details` : Enables collapsible content blocks.
- 31. `markdown_extensions.pydownx.superfences` : Enables nesting code blocks and other advanced features.
- 32. `plugins.search` : Enables the built-in search functionality.
- 33. `extra.generator` : Hides the "Made with Material for MkDocs" notice in the footer.

# Explanation

 2026-05-09  GitHub 



# Download

Download the complete documentation as a single PDF:

[Download PDF](#)

 2026-05-09  GitHub 

# About this project

## Overview

This project is a guide to **Material for MkDocs** documentation using the **Diátaxis framework**. It showcases how to organize content into the four pillars of Diátaxis: tutorials, how-to guides, reference, and explanations.

## Purpose

This is a **study project** applying the Diátaxis framework to Material for MkDocs.

## Credits

This documentation project is made possible by the contributions of several individuals and communities:

- **Diátaxis Framework:** Created by [Daniele Procida](#)
- **MkDocs:** Developed by the [MkDocs community](#)
- **Material for MkDocs:** Developed by [Martin Donath](#)

All original documentation, guidance, and frameworks are credited to their respective authors. This project serves as an educational implementation and study resource.



2026-05-09



GitHub

